

ASSIGNMENT – 6

API Rate Limiter using Token Bucket Algorithm (Spring Boot + Redis)

AIM

To implement an API rate limiter using the Token Bucket algorithm with Spring Boot and Redis. The system must enforce a controlled request rate per client while allowing short bursts up to a configured capacity, and should return HTTP 429 Too Many Requests when the bucket is empty.

SYSTEM ARCHITECTURE

The application is a Spring Boot 3.2.5 service with three REST endpoints. Client identity comes from an optional `client-id` header, falling back to the request IP. The `TokenBucketService` stores each client's token count and last-refill timestamp in Redis, so the limit is shared across instances if the service is scaled. The block diagram below shows the flow from the client request through the controller, service, Redis state, and HTTP response.

Fig 1. Rate limiter system architecture

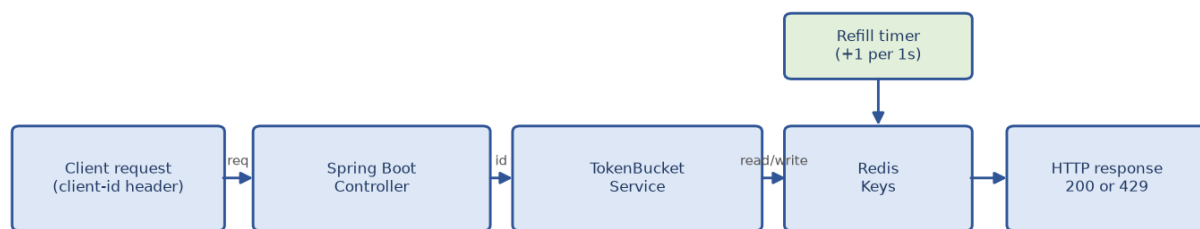


Fig 1. Rate limiter system architecture

TOKEN BUCKET ALGORITHM

Each client has a bucket that starts full. A request is allowed only if at least one token is available; the token is then consumed. Tokens are refilled at a fixed rate (refill-amount per refill-interval-millis) up to the bucket capacity. In this implementation Redis stores two keys per client: `rate\_limit:<client>:tokens` and `rate\_limit:<client>:last\_refill`. The `tryConsume` method is `synchronized` so that read-refill-decrement is atomic on a single instance.

CONFIGURATION

Property	Value	Effect
rate.limiter.bucket-capacity	10	Maximum tokens a client can accumulate

Property	Value	Effect
rate.limiter.refill-interval-millis	1000	One token added every 1 second
rate.limiter.refill-amount	1	Number of tokens added per refill interval
spring.data.redis.host	localhost	Redis server location
spring.data.redis.port	6379	Redis server port
server.port	8080	Spring Boot application port

ENDPOINTS

Method	Path	Description
GET	/api/health	Health check, not rate-limited
GET	/api/resource	Rate-limited resource; returns 200 if token available, 429 if not
GET	/api/rate-limit-status	Returns the remaining token count without consuming

CODE

```
package com.ssn.ratelimiter;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Service;

import java.util.concurrent.TimeUnit;

/**
 * Implements the Token Bucket algorithm for API rate limiting.
 *
 * Each client (identified by an ID or IP) gets its own "bucket" of tokens,
 * stored in Redis so the state is shared across all requests / instances.
 *
 * - The bucket starts full (bucketCapacity tokens).
 * - Every request tries to consume 1 token.
 * - Tokens are refilled at a fixed rate over time (refillAmount tokens
 *   every refillIntervalMillis milliseconds), up to the bucket capacity.
 * - If no tokens are available, the request is rejected (HTTP 429).
 *
 * Redis is used here (instead of a plain in-memory HashMap) so that the
```

```
* rate-limit state is centralized: if this application were scaled to
* multiple instances behind a load balancer, all instances would still
* share the same bucket for a given client.
*/
@Service
public class TokenBucketService {

    private final StringRedisTemplate redisTemplate;

    private final int bucketCapacity;
    private final long refillIntervalMillis;
    private final long refillAmount;

    private static final String TOKENS_SUFFIX = ":tokens";
    private static final String LAST_REFILL_SUFFIX = ":last_refill";
    private static final String KEY_PREFIX = "rate_limit:";

    public TokenBucketService(
        StringRedisTemplate redisTemplate,
        @Value("${rate.limiter.bucket-capacity:10}") int bucketCapacity,
        @Value("${rate.limiter.refill-interval-millis:1000}") long refillIntervalMillis,
        @Value("${rate.limiter.refill-amount:1}") long refillAmount) {
        this.redisTemplate = redisTemplate;
        this.bucketCapacity = bucketCapacity;
        this.refillIntervalMillis = refillIntervalMillis;
        this.refillAmount = refillAmount;
    }

    /**
     * Tries to consume one token for the given client.
     * Returns true if the request is allowed, false if the client is rate limited.
     *
     * synchronized is used here (instead of a Redis Lua script) to keep the
     * read -> refill -> decrement sequence atomic. This is enough for a
     * single-instance lab setup and is easy to explain: only one thread at a
     * time can update a given client's bucket.
     */
    public synchronized boolean tryConsume(String clientId) {
        refillBucket(clientId);

        long tokens = getTokens(clientId);
        if (tokens > 0) {
            redisTemplate.opsForValue().decrement(tokensKey(clientId));
            return true;
        }
        return false;
    }

    /** Returns the current token count for a client, applying any pending refill first. */
}
```

```
public long getRemainingTokens(String clientId) {
    refillBucket(clientId);
    return getTokens(clientId);
}

// ---- internal helpers ----

private void refillBucket(String clientId) {
    long now = System.currentTimeMillis();
    String lastRefillStr = redisTemplate.opsForValue().get(lastRefillKey(clientId));

    // First time we see this client: initialize a full bucket.
    if (lastRefillStr == null) {
        redisTemplate.opsForValue().set(tokensKey(clientId), String.valueOf(bucketCapacity));
        redisTemplate.opsForValue().set(lastRefillKey(clientId), String.valueOf(now));
        return;
    }

    long lastRefillTime = Long.parseLong(lastRefillStr);
    long elapsed = now - lastRefillTime;

    long intervalsPassed = elapsed / refillIntervalMillis;
    if (intervalsPassed <= 0) {
        return; // not time to refill yet
    }

    long currentTokens = getTokens(clientId);
    long newTokens = Math.min(bucketCapacity, currentTokens + (intervalsPassed * refillAmount));

    redisTemplate.opsForValue().set(tokensKey(clientId), String.valueOf(newTokens));
    // advance the "last refill" timestamp only by the whole intervals consumed,
    // so partial time isn't lost on the next check
    redisTemplate.opsForValue().set(lastRefillKey(clientId),
        String.valueOf(lastRefillTime + intervalsPassed * refillIntervalMillis));
}

private long getTokens(String clientId) {
    String tokensStr = redisTemplate.opsForValue().get(tokensKey(clientId));
    return (tokensStr != null) ? Long.parseLong(tokensStr) : bucketCapacity;
}

private String tokensKey(String clientId) {
    return KEY_PREFIX + clientId + TOKENS_SUFFIX;
}

private String lastRefillKey(String clientId) {
    return KEY_PREFIX + clientId + LAST_REFILL_SUFFIX;
}
}
```

CONTROLLER

```
package com.ssn.ratelimiter;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api")
public class RateLimiterController {

    private final TokenBucketService tokenBucketService;

    public RateLimiterController(TokenBucketService tokenBucketService) {
        this.tokenBucketService = tokenBucketService;
    }

    /** Health check endpoint - intentionally NOT rate limited. */
    @GetMapping("/health")
    public ResponseEntity<String> health() {
        return ResponseEntity.ok("Service is healthy");
    }

    /**
     * Rate-limited endpoint. Client is identified by the "client-id" header
     * if present, otherwise falls back to the request's IP address.
     */
    @GetMapping("/resource")
    public ResponseEntity<String> resource(
        @RequestHeader(value = "client-id", required = false) String clientId,
        HttpServletRequest request) {

        String identifier = resolveClientId(clientId, request);

        if (tokenBucketService.tryConsume(identifier)) {
            return ResponseEntity.ok("Request processed successfully");
        } else {
            return ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS)
                .body("HTTP 429 Too Many Requests: rate limit exceeded");
        }
    }
}
```

```
    }  
}  
  
/** Lets you check remaining tokens without consuming one. */  
@GetMapping("/rate-limit-status")  
public ResponseEntity<String> rateLimitStatus(  
    @RequestHeader(value = "client-id", required = false) String clientId,  
    HttpServletRequest request) {  
  
    String identifier = resolveClientId(clientId, request);  
    long tokens = tokenBucketService.getRemainingTokens(identifier);  
    return ResponseEntity.ok("Remaining tokens: " + tokens);  
}  
  
private String resolveClientId(String clientId, HttpServletRequest request) {  
    if (clientId != null && !clientId.isBlank()) {  
        return clientId;  
    }  
    String ip = request.getRemoteAddr();  
    return (ip != null) ? ip : "unknown";  
}  
}
```

TEST OUTPUT

The application was built and run on a Windows host using Maven 3.9.16 and Java 21, with Redis provided by Memurai on localhost:6379. The `test-rate-limiter.bat` script was executed with 15 requests for `client-id: user1`. Because the default bucket capacity is 10 and one token refills during the burst, the first 11 requests returned HTTP 200 and the remaining 4 were rejected with HTTP 429. This confirms the token bucket allows a short burst above the sustained rate and then enforces the limit.

```
Sending 15 requests as client-id: user1  
-----  
Request 1: 200 OK  
Request 2: 200 OK  
Request 3: 200 OK  
Request 4: 200 OK  
Request 5: 200 OK  
Request 6: 200 OK  
Request 7: 200 OK  
Request 8: 200 OK  
Request 9: 200 OK  
Request 10: 200 OK  
Request 11: 200 OK  
Request 12: 429 TOO MANY REQUESTS
```

```
Request 13: 429 TOO MANY REQUESTS
Request 14: 429 TOO MANY REQUESTS
Request 15: 429 TOO MANY REQUESTS
-----
```

Summary: 11 succeeded, 4 rate-limited

Current bucket status:

Remaining tokens: 0

Fig 3. Burst test result: 15 rapid requests with client-id "user1"

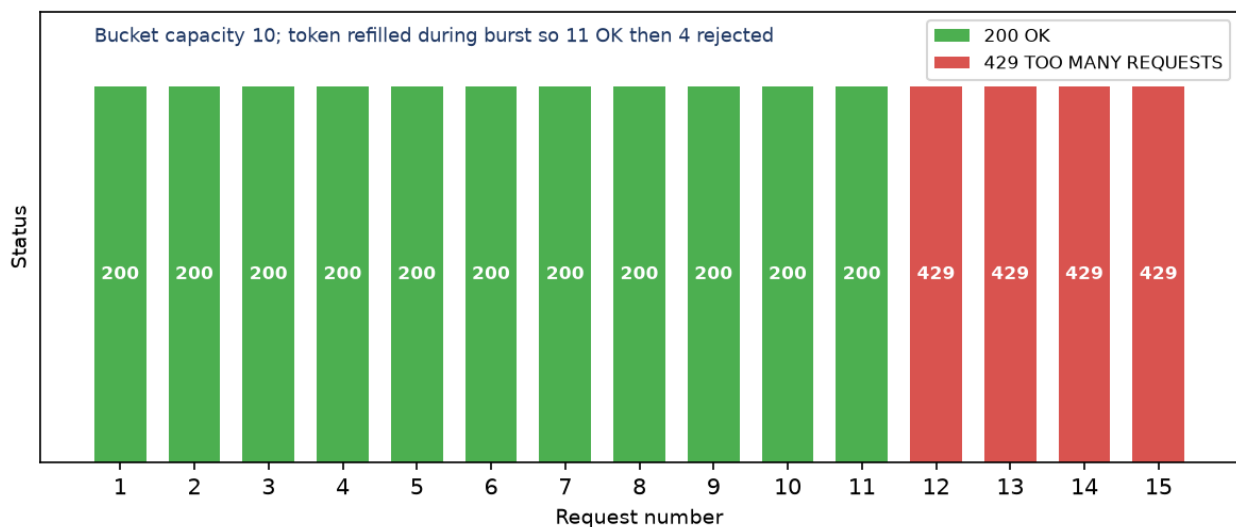


Fig 3. Burst test: 15 rapid requests

VERIFICATION

- `/api/health`` returned 200 with body ``Service is healthy``.
- 15 rapid requests to `/api/resource`` returned 11×200 and 4×429 , matching the token bucket behavior.
- `/api/rate-limit-status`` for ``user1`` returned ``Remaining tokens: 0`` immediately after the burst, and later refilled to 2 after a few seconds.
- Client ``user1`` and ``user2`` have independent buckets; running the test with a new ``client-id`` resets the test because Redis keeps per-client keys.
- Redis keys ``rate_limit:<client>:tokens`` and ``rate_limit:<client>:last_refill`` are visible with ``keys rate_limit:*``, proving the state is centralized.

LEARNING OUTCOMES

- Implemented the Token Bucket rate-limiting algorithm in a Spring Boot service.
- Used Redis as the shared state store so rate limits are correct under horizontal scaling.
- Observed the difference between burst capacity (bucket size) and sustained rate (refill rate).
- Returned the standard HTTP 429 status code for rejected requests.
- Understood why ``synchronized`` is used for atomic read-refill-decrement in a single-instance lab setup.

REFERENCES

- Spring Boot 3.2.5 Reference Documentation — <https://spring.io/projects/spring-boot>
- Redis Documentation — <https://redis.io/documentation>
- RFC 6585: HTTP Status Code 429 (Too Many Requests)
- System Design course, SSN College of Engineering, Academic Year 2026-27.